Event Ticketing Platform — Day-by-Day 8-Week Schedule

A detailed execution plan to pair with the project spec. Assumes ~5 working days/week, full-time focus.

Sequencing strategy (read this first)

Backend first, frontend second — not parallel. Here's the reasoning, so he understands it rather than just following it:

1. **The senior signal is in the backend** — concurrency, observability, AWS, AI architecture. That's where the time should go, and it should be done well before attention splits.
2. **The frontend consumes finished APIs.** Building UI against half-built, still-changing endpoints means constant rework. Stabilize the contract first.
3. **Splitting focus early is a junior trap.** One person doing both at once usually does neither well. Finish the hard thing, then layer the UI on top.

So: **Weeks 1–7 are backend + infra + AI + load testing. Frontend is concentrated in Week 8** (with an optional early thin slice if the role is genuinely full-stack — noted below). If the role is backend-focused, a lighter frontend is completely fine and more honest.

The discipline that runs through all 8 weeks: keep the **decision log** ("chose X over Y because Z") and write a **post-mortem** after every chaos drill. These two artifacts are what make him sound senior.

WEEK 1 — Design & foundation

Goal: a written design, a running skeleton app, and the data model.

- **Day 1 — Design doc.** Write it before any code. Problem statement, the three user types, the core flows, and the architecture diagram (the AWS diagram from the spec). For every major choice, write the "why" in the decision log. Set up the GitHub repo.
- **Day 2 — Local environment.** Spring Boot 3 / Java 21 project scaffolded. PostgreSQL + Redis running locally via Docker Compose. App boots, connects to DB, exposes a `/health` endpoint.
- **Day 3 — Data model part 1.** Define and migrate `users`, `events`, `ticket_types` (with `available_quantity` + `version`). Use Flyway or Liquibase for migrations from day one.
- **Day 4 — Data model part 2.** `orders`, `order_items`, `tickets`. Wire up JPA entities and repositories. Seed some test data.
- **Day 5 — Review + buffer.** Verify schema, clean up, write the first few unit tests to establish the testing habit. Catch up if behind.

End of week: running app, full schema, design doc + diagram committed.

WEEK 2 — Core APIs & auth

Goal: the CRUD surface and security in place.

- **Day 1 — Auth.** `POST /auth/register`, `POST /auth/login` returning JWT. Spring Security config.
- **Day 2 — Role-based access.** Attendee / organizer / admin roles; route guards on the server side. Verify a non-organizer can't create events.
- **Day 3 — Event APIs.** `GET /events`, `GET /events/{id}`, `POST /events`, `PUT /events/{id}`. Validation on inputs.
- **Day 4 — Ticket-type + read APIs.** Ticket types under events; `GET /me/tickets`; `GET /events/{id}/sales` stub.
- **Day 5 — Tests + cleanup.** Unit + integration tests for auth and event APIs. Decision-log update.

End of week: a secured, testable API for everything *except* the purchase flow.

WEEK 3 — Purchase flow & concurrency (the centerpiece)

Goal: witness the oversell bug, then fix it properly.

- **Day 1 — Naive purchase flow.** Build `POST /orders` the obvious (wrong) way: read availability, decrement, save. This is intentionally buggy.
- **Day 2 — Witness the bug.** Write a quick concurrent test (or a first k6 burst) that fires many simultaneous buys at a tiny inventory. Watch it oversell. Document it in the decision log — this is a key learning moment.
- **Day 3 — Optimistic locking.** Add the `version` column logic; update fails on conflict; retry-or-reject. Re-test → exactly the right number sold.
- **Day 4 — Pessimistic + distributed lock.** Implement `SELECT ... FOR UPDATE`, then a Redis distributed lock. Understand when each is appropriate.
- **Day 5 — Idempotency + payment.** `Idempotency-Key` header so double-clicks don't double-charge; integrate Stripe test mode; `POST /orders/{id}/pay`. Correct transaction boundaries.

End of week: a purchase flow that sells exactly the right number under concurrency, with idempotent payment. The hardest part is done.

WEEK 4 — Async layer

Goal: move side-effects off the request path, reliably.

- **Day 1 — SQS integration.** On successful purchase, publish a message instead of doing email work inline.
- **Day 2 — Worker/consumer.** A consumer that processes purchase-confirmation messages.
- **Day 3 — Email via SES.** Generate the ticket + QR code, send the confirmation email from the worker.
- **Day 4 — Retries + dead-letter queue.** Configure retries and a DLQ for messages that keep failing.
- **Day 5 — Organizer stats + tests.** Update sales numbers asynchronously; test the full async path end-to-end.

End of week: purchases trigger async confirmation reliably, with a DLQ safety net.

WEEK 5 — Containerize, infra-as-code, deploy

Goal: it runs on AWS, deployed by a pipeline.

- **Day 1 — Docker.** Dockerize the app; multi-stage build; push to ECR.
- **Day 2 — Terraform part 1.** VPC, subnets, security groups, IAM roles, RDS, ElastiCache.
- **Day 3 — Terraform part 2.** ECS/Fargate service, ALB/API Gateway, SQS, S3, SES, Secrets Manager (DB creds, Stripe key, LLM key).
- **Day 4 — First deploy.** Get it running on Fargate end-to-end against real RDS/Redis/SQS. Debug the inevitable networking/IAM issues (great learning).
- **Day 5 — CI/CD.** GitHub Actions: build → test → push image → deploy on merge. Add a billing alarm.

End of week: a live, deployed app shipped by a pipeline, infra fully in Terraform.

WEEK 6 — Observability + start RAG

Goal: full visibility, and the AI foundation laid.

- **Day 1 — Structured logging + correlation IDs** threaded through app and worker.
- **Day 2 — Metrics + dashboards.** Latency, error rate, throughput, DB pool usage, SQS queue depth, JVM heap/GC → CloudWatch (or Prometheus + Grafana).
- **Day 3 — Alarms + tracing.** Error-rate alarm, queue-depth alarm; distributed tracing via correlation IDs.

- **Day 4 — RAG ingestion.** Enable pgvector; build the ingestion pipeline (chunking + embeddings) over event/venue/FAQ/policy content using Spring AI.
- **Day 5 — RAG retrieval.** Top-k retrieval with similarity threshold; verify relevant chunks come back.

End of week: the system is observable, and the RAG pipeline retrieves correctly.

WEEK 7 — Finish RAG + load testing & chaos drills

Goal: senior-grade AI feature, and proof it all holds under fire.

- **Day 1 — Grounded generation.** `POST /events/{id}/ask` — assemble prompt with context, cite sources, return "I don't know" on weak retrieval. Add semantic search endpoint.
- **Day 2 — AI hardening.** Cache embeddings/responses, track tokens/cost, rate-limit AI endpoints, prompt-injection guardrail, graceful fallback when LLM is down. Build a small eval set.
- **Day 3 — Load test suite.** Write k6 scripts: oversell burst, spike (10→2,000), soak, sustained. Run the **oversell before/after** as the headline demo.
- **Day 4 — Chaos drills round 1.** Run drills 2–6 (queue backup, pool exhaustion, memory leak, cascading timeout, poison message). One post-mortem each.
- **Day 5 — Chaos drills round 2.** Drills 7–12 (bad deploy/rollback, config drift, and the four AI drills). Post-mortems each.

End of week: a hardened AI feature + a folder of post-mortems = real incident stories.

WEEK 8 — Frontend + final hardening + rehearsal

Goal: a working Angular frontend and a candidate who can explain everything.

- **Day 1 — Angular setup + auth.** Project scaffold, Angular Material, routing skeleton, login/signup, JWT handling, route guards, API service layer (HttpClient + RxJS).
- **Day 2 — Attendee buy-flow** (highest priority — it showcases the concurrency work): browse/search, event detail (with AI chat widget), checkout (reactive form), confirmation, my-tickets.
- **Day 3 — Organizer dashboard.** Sales summary cards, sales-over-time chart, create/edit event (reactive form), per-event detail. Minimal admin tables.
- **Day 4 — Final hardening.** Fix loose ends, tighten error handling, verify the full end-to-end flow on the deployed environment, polish README + architecture diagram.
- **Day 5 — Rehearsal.** Out loud: the oversell story, the AWS choices, the RAG tradeoffs (why RAG over fine-tuning), and a walk through 2–3 post-mortems. Mock system-design question. This is what converts the build into interview performance.

End of week: a complete, deployed, full-stack system + a candidate who can narrate every decision.

Optional adjustment if the role is genuinely full-stack

If Angular is a real requirement (not just "nice to have"), insert a **thin frontend slice at the end of Week 2** — just login + browse + event detail against the early APIs — so he has more total Angular reps. Keep the heavy lift in Week 8. Don't move it earlier than that; the backend still comes first.

Milestone summary

By end of…	You should have
Week 1	Design doc, diagram, schema, running skeleton
Week 2	Secured CRUD APIs + auth
Week 3	**Concurrency-safe purchase flow** (the centerpiece)
Week 4	Reliable async confirmation + DLQ
Week 5	**Deployed on AWS via CI/CD**, infra in Terraform
Week 6	Full observability + RAG retrieval working
Week 7	**Hardened RAG + load tests + post-mortems**
Week 8	**Working Angular frontend + rehearsed interview narrative**

If he falls behind (cut in this order)

Protect the backend signal. If time runs short, trim from the *bottom* of this list, not the top:

1. **Never cut:** the concurrency flow (Week 3), deployment (Week 5), observability + one or two chaos drills (Weeks 6–7). These are the core senior signals.
2. Trim first: frontend polish (keep the buy-flow, drop admin UI refinement).
3. Then: reduce the AI feature to RAG-only (drop NL analytics).
4. Then: fewer chaos drills (keep oversell, queue backup, and one AI drill).

A smaller project done *well and understood deeply* beats a bigger one he can't explain.